

metaDEBASS: Trust-Aware Early-Epoch Transient Pipeline

Repository note

April 30, 2026

Abstract

If you are joining metaDEBASS for the hackathon, this is the map I would want in front of me before opening files at random. It explains the actual science product, how the repository is organized, how the trust-aware pipeline moves from raw broker payloads to scored outputs, where the older baseline still fits, and what still blocks the full science goal. This update was checked against the repository contracts, the current feature extractor, the expert registry, the SCC guide, and the latest checked-in reports as of April 30, 2026.

Contents

1	What metaDEBASS Is Trying to Do	2
2	Science Contract	3
2.1	Primary Output	3
2.2	No-Future-Leakage Epochs	3
2.3	Temporal Semantics Must Survive Normalization	3
2.4	Truth Must Stay Separate from Broker Outputs	3
2.5	Unsafe ALeRCE History	3
3	High-Level Repository Map	4
3.1	A Small but Important Reality Check	4
4	The Core Python Package	4
4.1	access/: Broker Adapters	4
4.2	features/: Early-Epoch Lightcurve Features	5
4.3	ingest/: Bronze, Silver, Gold, and Baseline Epoch Tables	5
4.4	projectors/: Per-Expert Feature Mapping	6
4.5	experts/local/: Local Expert Reruns	7
5	Data Products and Their Meaning	7
5.1	Temporal Exactness Labels	8
6	Trust-Aware Pipeline Walkthrough	8
6.1	Stage 1: Collect Weak Seed Objects and Lightcurves	8
6.2	Stage 2: Backfill Broker Outputs to Bronze	9
6.3	Stage 3: Normalize Bronze to Silver	9
6.4	Stage 4: Build Truth and Gold Tables	9
6.5	Stage 5: Run Local Experts	9

6.6	Stage 6: Train Per-Expert Trust Heads	9
6.7	Stage 7: Train the Follow-Up Head	10
6.8	ML Algorithm Details	10
6.9	Stage 8: Emit the Final Payload	11
7	Baseline Benchmark Path	11
8	Registered Experts and Coverage	11
9	How to Run the Project	13
9.1	Local Smoke Test	13
9.2	BU SCC Execution Model	13
9.3	SCC Bootstrap Note	13
10	Tests and What They Tell You	15
11	Current Repository Status on April 30, 2026	15
11.1	What Is Already Strong	16
11.2	What Still Blocks the Full Science Goal	16
11.3	Checked-In Metrics Snapshot	16
11.4	How To Interpret That Status	17
12	Recommended Reading Order for a New Teammate	17
13	Bottom Line	17

1 What metaDEBASS Is Trying to Do

metaDEBASS is asking a slightly different question from a normal transient classifier. Instead of forcing every signal into one final class as early as possible, it first asks which expert looks reliable at the current stage of the lightcurve. For one object at one alert epoch, the practical questions are:

- which expert is available right now,
- which expert should be trusted right now, and
- what evidence that expert is contributing.

That is why the primary scored object is `expert_confidence`: calibrated per-expert confidence at `(object_id, n_det, alert_jd)`. A trust-weighted follow-up proxy is still useful, but it is downstream of the trust layer and should not be described as the primary science product.

Conceptually, the trust-aware path is:

```
lightcurves + broker payloads
-> bronze
-> silver broker_events
-> gold object_epoch_snapshots
-> gold expert_helpfulness
-> per-expert trust heads
-> optional follow-up head
-> score payload with expert_confidence
```

The baseline path is still useful as a benchmark, but it is not the main science story here.

2 Science Contract

After reading through the repo, these are the rules that keep showing up. They are the guardrails. If we ignore them, the code may still run, but the science claim gets much weaker.

2.1 Primary Output

The primary science output is `expert_confidence`: calibrated per-expert trust at a specific epoch:

$$q_{\{e,t\}} = P(\text{expert } e \text{ is trustworthy now} \mid \text{current epoch evidence})$$

The secondary output is a trust-weighted follow-up proxy score. It is useful operationally, but it must sit downstream of the trust layer.

2.2 No-Future-Leakage Epochs

For an object with N detections, the pipeline builds rows for $n_{det} = 1, 2, \dots, N$. The row at epoch k can only use detections $1 \dots k$. The same idea applies to expert evidence: if an expert event is supposed to be historically safe, it has to satisfy `event_time_jd <= alert_jd`.

2.3 Temporal Semantics Must Survive Normalization

metaDEBASS carries event-time metadata all the way through the pipeline. That may sound fussy at first, but it is one of the most important design choices in the repo. The fields that matter are:

- `event_time_jd`
- `n_det`
- `alert_id`
- `event_scope`
- `temporal_exactness`

This is what lets the trust-aware pipeline tell the difference between genuinely epoch-safe history and a backfilled object snapshot that only looks historical.

2.4 Truth Must Stay Separate from Broker Outputs

Weak labels are allowed for bootstrapping, but the repo is pretty clear that they must stay labeled as weak. In other words, `data/labels.csv` is a seed set, not the thing we should eventually cite as truth. The file meant to hold the canonical truth layer is `data/truth/object_truth.parquet`.

2.5 Unsafe ALeRCE History

ALeRCE probabilities in this repo are usually object-level snapshots, not archived per-alert scores. That is why historical backfills get marked `latest_object_unsafe` and dropped from trust training by default.

3 High-Level Repository Map

If you only have a few minutes to get your bearings, Table 1 is the fastest way in.

Table 1: Top-level repository guide.

Path or area	Purpose
README.md	Main project summary, trust-aware workflow, baseline workflow, and output expectations.
docs/science_contract.md	Compact statement of the science rules the code is supposed to respect.
src/debass_meta/access/	Adapters for broker APIs and fixture fallback.
src/debass_meta/features/	No-leakage lightcurve feature extraction.
src/debass_meta/ingest/	Bronze, silver, gold, and baseline epoch-table builders.
src/debass_meta/ projectors/	Expert-specific projection logic into common trust features.
src/debass_meta/experts/ local/	Local expert wrappers for rerunnable epoch-safe experts, including SuperNNova, ParSNIP, ALeRCE-LC, SALT3, light-curve feature, AMPEL/SNGuess, and ORACLE paths.
scripts/	Most executable pipeline logic lives here today.
jobs/	BU SCC submission scripts and resume helpers.
inventory/	Human-readable broker and expert metadata.
schemas/	JSON schema files for the canonical data products.
tests/	Tests that define many of the intended contracts, especially no-leakage and exactness behavior.
codex_build_brief/	Design brief describing the intended phase-1 trust-aware architecture.

3.1 A Small but Important Reality Check

The canonical package name is now `debass_meta`, and the main source tree lives under `src/debass_meta/`. There is still a compatibility shim for the older `debass_meta_meta` name, but teammates should treat that as legacy support rather than the primary package layout.

4 The Core Python Package

4.1 access/: Broker Adapters

The access layer is where the repo talks to outside services, or falls back to cached fixtures when those services are missing or unavailable.

- `alerce.py`: queries ALeRCE probabilities and lightcurves. Light-curve probabilities from current object endpoints are marked `latest_object_unsafe`; stamp classifiers are treated as `static_safe` when they are first-alert image context.
- `fink.py`: queries Fink alert history and keeps alert-level fields such as `rf_snia_vs_nonia`, `snn_snia_vs_nonia`, `snn_sn_vs_all`, and SLSN-related scores. These are exact alert-level events when the alert timestamp is present.
- `lasair.py`: fetches Sherlock context labels. These are treated as context tags, not calibrated posteriors.

- `babamul.py`: ingests BOOM/Babamul-style contaminant and cross-survey flags. Its rows are `static_safe` context.
- `pitt.py`: reads Pitt-Google broker outputs, including LSST SuperNNova rows with time-stamped event semantics when available.
- `antares.py` and `ampel.py`: integration paths for ANTARES and AMPEL expert outputs.
- `rubin_rsp.py`: helper access layer for Rubin Science Platform data probes.
- `tns.py`: handles TNS credentials, rate-limited API access, type mapping, and the truth-side crossmatch support used to build stronger labels.
- `alerce_history.py`: turns ALERCE object snapshots into per-epoch records while explicitly preserving the fact that those rows are unsafe for historical trust training.

The common adapter contract is `BrokerOutput` in `access/base.py`. It keeps the raw payload, a semantic type, optional extracted field rows, and the usual provenance details like endpoint, request parameters, and whether fixture fallback was used.

4.2 `features/`: Early-Epoch Lightcurve Features

This is one of the cleaner parts of the codebase and worth reading closely. The main feature extractor is `features/lightcurve.py`, and it computes 51 features from a truncated lightcurve slice only. The current feature list includes:

- detection counts: `n_det` plus per-band counts for `u`, `g`, `r`, `i`, `z`, and `y`,
- timing: `t_since_first`, `t_baseline`,
- brightness summaries: first, last, min, mean, standard deviation, range,
- rise or decline summaries: `dmag_dt` and `dmag_first_last`,
- per-band brightness summaries for the six Rubin bands,
- color features for `g-r`, `r-i`, `i-z`, and `z-y`, plus slopes,
- quality summary: `mean_quality`, with `mean_rb` retained as a legacy alias,
- survey flag: `survey_is_lsst`.

The authoritative column list is `FEATURE_NAMES`. The key helper is `extract_features_at_each_epoch()`. It emits one row per detection count and makes sure each row only sees the detections available at that point in time.

4.3 `ingest/`: Bronze, Silver, Gold, and Baseline Epoch Tables

Bronze. `ingest/bronze.py` writes append-only parquet partitions containing raw broker responses plus extracted field or event breakdowns. This layer is mostly about preservation: keep what came back from the broker, and do not overwrite it later.

Silver. `ingest/silver.py` normalizes bronze payloads into `data/silver/broker_events.parquet`. Each row is an expert-field event with:

- object and broker identifiers,
- semantic type,
- expert key,
- event scope,
- event time,
- exactness label,
- projected numeric score where meaningful, and
- provenance metadata.

This is the layer where the project stops flattening everything into a generic score table. The schema keeps enough context to remember what each broker field actually meant.

Gold v2. `ingest/gold.py` builds `object_epoch_snapshots.parquet`. This is really the center of the trust-aware feature path. It joins truncated lightcurve features with the latest event-safe expert evidence for each phase-1 expert at each epoch.

Its helper `select_events_asof()` enforces the main timing logic:

- use rows with `event_time_jd <= alert_jd` when available,
- otherwise allow `static_safe` context rows,
- otherwise allow `rerun_exact` local expert rows,
- only include `latest_object_unsafe` rows if the caller explicitly opts in.

Baseline epoch table. `ingest/epochs.py` and `scripts/build_epoch_table_from_lc.py` support the older fused-classifier benchmark path. That branch still respects no-future-leakage for the lightcurve, but it is solving a different problem from the trust-aware gold tables.

4.4 projectors/: Per-Expert Feature Mapping

The projector layer does the translation work. It turns very different expert outputs into trust-model features without pretending they all started in the same probability space.

- `fink.py` and `fink_lsst.py`: project Fink ZTF and LSST outputs, including SN-filter experts whose target is SN-vs-other rather than ordinary top-class correctness.
- `alerce.py`: aggregates ALERCE class probabilities into the repository ternary label space.
- `lasair.py`: emits one-hot context features for Sherlock tags.
- `babamul.py`: projects contaminant and cross-survey context flags.

- `pittgoogle.py`, `antares.py`, and `ampel.py`: normalize broker-specific expert families into the same contract.
- `local.py`, `local_salt3.py`, `local_lc_features.py`, and `local_oracle.py`: map local reruns and physics/feature experts into trust features.

The full scoring registry is `ALL_EXPERT_KEYS` in `projectors/base.py`; this checkout currently registers 28 expert or context keys. The older `PHASE1_EXPERT_KEYS` list remains as a backward-compatible eight-key subset for tests and legacy scripts.

```
fink/snn
fink/rf_ia
fink/slsn
parsnip
supernnova
alerce_lc
alerce/lc_classifier_transient
alerce/stamp_classifier
... plus ALERCE BHRF/ATAT, Fink LSST, Lasair,
Babamul, Pitt-Google, ANTARES, AMPEL, SALT3,
LC-feature, and ORACLE experts
```

4.5 experts/local/: Local Expert Reruns

This directory contains wrappers for rerunning local experts on truncated archival lightcurves. It is important because local reruns can produce `rerun_exact` evidence when broker history is not available.

- `supernnova.py`: epoch-aware SuperNNova wrapper; the SCC guide treats this as a CPU batch stage when the Fink-style artifacts are installed.
- `parsnip.py`: ParSNIP wrapper. It expects both `model.pt` and `classifier.pkl`; GPU is mainly relevant for this stage when real weights are available.
- `alerce_lc.py`: local ALERCE-style LightGBM light-curve expert, used as the safe substitute for unsafe ALERCE object snapshots.
- `salt3_fit.py` and `lc_features.py`: local physics/feature experts that can be rerun at each epoch.
- `snguess.py` and `ampel_runner.py`: AMPEL/SNGuess integration paths.
- `base.py`: defines the common local expert interface and `ExpertOutput` container.

5 Data Products and Their Meaning

For a new teammate, the easiest way to understand the system is to follow the files it is trying to produce.

Table 2: Canonical data products in the trust-aware path.

Artifact	Meaning
<code>data/labels.csv</code>	Weak seed object list and weak ternary labels. Useful for bootstrapping, not final truth.
<code>data/lightcurves/*.json</code>	Cached per-object photometry used to build truncated epoch slices.
<code>data/tns_public_objects.csv</code>	Optional TNS bulk catalog used for stronger offline crossmatching.
<code>data/bronze/*.parquet</code>	Raw append-only broker payload archive.
<code>data/silver/broker_events.parquet</code>	Event-level normalized broker rows with exactness and provenance semantics.
<code>data/silver/local_expert_outputs/<expert>/</code>	Local rerun outputs by expert and epoch.
<code>data/truth/object_truth.parquet</code>	Canonical truth table. Intended to separate real truth from broker-derived weak labels.
<code>data/gold/object_epoch_snapshots.parquet</code>	The main trust-aware feature table, one row per object and epoch.
<code>data/gold/expert_helpfulness.parquet</code>	One row per available expert at each epoch, with correctness and usefulness targets.
<code>models/trust/<expert>/</code>	Per-expert trust models and metadata.
<code>models/followup/</code>	Optional follow-up proxy model.
<code>reports/scores/*.jsonl</code>	Final trust-aware score payloads.

5.1 Temporal Exactness Labels

The repo uses a small vocabulary for event safety:

- `exact_alert`: truly timed alert-level evidence,
- `static_safe`: static context that is safe to reuse historically,
- `rerun_exact`: local rerun performed on the truncated lightcurve,
- `latest_object_unsafe`: latest object snapshot that is not historically epoch-safe.

If a teammate only remembers one metaDEBASS-specific idea after skimming the code, it should probably be this exactness system.

6 Trust-Aware Pipeline Walkthrough

6.1 Stage 1: Collect Weak Seed Objects and Lightcurves

`scripts/download_alerce_training.py` queries ALerCE for high-probability examples from several transient classes, fetches lightcurves, and writes:

- `data/labels.csv`
- `data/lightcurves/<object>.json`

This is a bootstrap step, not a truth pipeline. It gives the project something to work with, but the labels it writes are still weak self-labels.

6.2 Stage 2: Backfill Broker Outputs to Bronze

`scripts/backfill.py` instantiates the enabled adapters and fetches one raw response per broker per object. Those responses land in append-only bronze partitions.

6.3 Stage 3: Normalize Bronze to Silver

`scripts/normalize.py` calls the silver transform to create `broker_events.parquet`. This is where raw broker payloads become event-level rows with explicit timing semantics and expert keys.

6.4 Stage 4: Build Truth and Gold Tables

The truth layer is broader than it used to be. `scripts/build_truth_table.py` can now merge three sources in priority order: curated external truth, TNS crossmatch products, and weak-label fallback from `labels.csv`. If you want stronger labels, the relevant helpers are `scripts/download_tns_bulk.py` and `scripts/crossmatch_tns.py`.

Three scripts still define the core trust-aware training inputs once truth exists:

- `scripts/build_truth_table.py`
- `scripts/build_object_epoch_snapshots.py`
- `scripts/build_expert_helpfulness.py`

Together they establish:

- the target class and follow-up proxy,
- the per-epoch feature table,
- the per-expert trust target rows.

6.5 Stage 5: Run Local Experts

`scripts/local_infer.py` loops over each object and each detection count, truncates the lightcurve, and asks each local expert for an epoch-specific prediction. This is why local experts matter so much in metaDEBASS: unlike unsafe object snapshots, they can at least in principle be rerun exactly at the historical epoch.

At the code level, the local expert pool includes SuperNNova, ParSNIP, ALerCE-LC, SALT3- χ^2 , a Bazin/Villar-style light-curve feature head, AMPEL/SNGuess, and ORACLE paths. Which ones produce real rows depends on installed packages and model artifacts. The important contract is that any local row emitted for historical training must be produced from the truncated lightcurve and marked `rerun_exact`.

6.6 Stage 6: Train Per-Expert Trust Heads

The intended trust training script is `scripts/train_expert_trust.py`. Its job is to learn one trust model per expert, using `expert_helpfulness.parquet` as the supervision layer and grouped object splits to avoid leakage across epochs of the same object. Most experts train against `is_topclass_correct`; SN-filter experts such as `fink/slsn`, `fink_lsst/snn`, `fink_lsst/cats`, and `ampel/snguess` train against `is_sn` because their projections are not ordinary Ia-vs-non-Ia posteriors.

This is where the project idea shows up most directly. The question is not just “what class is this object,” but “is this expert likely to be useful right now.”

6.7 Stage 7: Train the Follow-Up Head

`scripts/train_followup.py` is the secondary stage. It is meant to consume trust features and projected expert evidence to estimate `p_follow_proxy`.

6.8 ML Algorithm Details

All classifiers (trust heads, follow-up head, and the baseline `EarlyMetaClassifier`) use **LightGBM** gradient-boosted trees (Ke et al. 2017, NeurIPS). The switch from `RandomForest` was motivated by three properties:

1. **Native NaN handling.** Mixed broker and mixed-survey tables are sparse by design: LSST-only experts are NaN for ZTF objects, ZTF-only experts are NaN for LSST objects, and many broker classifiers trigger only for subsets of alerts. `RandomForest` required median imputation, which is lossy. `LightGBM` learns the optimal split direction for missing values during tree construction, meaning the *pattern of missingness itself* becomes an informative signal.
2. **Leaf-wise tree growth.** The heterogeneous feature space (broker probabilities in $[0, 1]$, magnitudes in $[15, 22]$, time spans in days) benefits from leaf-wise splitting, which more efficiently captures interactions such as “ALeRCE says Ia but the color evolution disagrees.”
3. **Early stopping.** The `EarlyMetaClassifier` trains up to 1000 boosting rounds with early stopping (patience 50) on a held-out calibration set, preventing overfitting on the small smoke-test datasets while allowing full capacity on production runs with 2000+ objects.

Shared regularisation across all models:

```
learning_rate    = 0.05
num_leaves       = 31
min_child_samples= 5
feature_fraction = 0.8    (column subsampling)
bagging_fraction = 0.8    (row subsampling)
reg_alpha        = 0.1    (L1)
reg_lambda       = 0.1    (L2)
```

Binary models (trust heads, follow-up) set `is_unbalance=True` for automatic class-weight correction. The multiclass baseline uses explicit per-object inverse-frequency sample weights.

Calibration: the baseline uses temperature scaling on a separate calibration split; the binary models can use isotonic regression via `calibrate.py:IsotonicCalibrator`.

Data splits follow an object-level grouping strategy (`GroupSplit` or `GroupKFold`) so that all epochs of the same transient stay in the same fold, preventing train/test contamination through repeated observations of the same object.

6.9 Stage 8: Emit the Final Payload

`scripts/score_nightly.py` assembles a JSON payload shaped roughly like:

```
{
  "object_id": "...",
  "n_det": 4,
  "alert_jd": ...,
  "expert_confidence": {
    "fink/snn": {
      "available": true,
      "trust": 0.84,
      "prediction_type": "class_correctness",
      "mapped_pred_class": "snia",
      "projected": {...}
    }
  },
  "ensemble": {
    "p_follow_proxy": ...,
    "recommended": ...
  }
}
```

This payload is the cleanest expression of the whole project: show trust per expert first, then roll that into an operational recommendation if you want one.

7 Baseline Benchmark Path

The repository still contains the earlier fused baseline workflow:

- `scripts/build_epoch_table_from_lc.py`
- `scripts/train_early.py`
- `scripts/score_early.py`

This path produces a single early-epoch LightGBM classifier. It is worth knowing because it gives you a sanity-check comparator, but it is still the side branch here, not the main architecture. Since the April 2026 upgrade, it uses the same LightGBM backend as the trust-aware path, with early stopping and temperature-scaled calibration.

8 Registered Experts and Coverage

The expert registry has grown beyond the original phase-1 list. As of this update, `EXPERT_REGISTRY` defines 28 expert or context keys. Not all of them are active in every run: the registry is the contract for what the gold builder and scorer know how to represent, while actual coverage depends on broker availability, survey, credentials, and local expert artifacts.

Table 3: Registered expert families in `src/debass_meta/projectors/base.py`.

Family	Expert keys	Timing and role
Fink ZTF	<code>fink/snn</code> , <code>fink/rf_ia</code> , <code>fink/slsn</code>	Per-alert <code>exact_alert</code> signals when alert history is available. SLSN is treated as an SN-filter expert, not an Ia posterior.
Local reruns	<code>parsnip</code> , <code>supernnova</code> , <code>alerce_lc</code>	<code>rerun_exact</code> experts when the local model artifacts are installed and inference is run at each truncated epoch.
ALeRCE ZTF	<code>alerce/lc_classifier_transient</code> , <code>alerce/stamp_classifier</code> , <code>alerce/lc_classifier_BHRF_forced_phot_transient</code> , <code>alerce/lc_classifier_BHRF_forced_phot_top</code> , <code>alerce/LC_classifier_ATAT_forced_phot(beta)</code> , <code>alerce/stamp_classifier_2025_beta</code>	Light-curve object snapshots are <code>latest_object_unsafe</code> unless archived safely. Stamp rows can be <code>static_safe</code> when used as first-alert image context.
ALeRCE Rubin	<code>alerce/stamp_classifier_rubin_beta</code>	Rubin/LSST stamp context key; coverage depends on ALeRCE Rubin beta availability.
Fink LSST	<code>fink_lsst/snn</code> , <code>fink_lsst/cats</code> , <code>fink_lsst/early_snia</code>	LSST alert-level Fink outputs. These are designed for mixed-survey training with missing ZTF-only columns left as NaN.
Context brokers	<code>lasair/sherlock</code> , <code>babamul</code>	Static or contextual evidence, not class posteriors. Useful for host, contaminant, and crossmatch context.
Pitt-Google	<code>pittgoogle/supernnova_lsst</code> , <code>pittgoogle/supernnova_ztf</code> , <code>pittgoogle/upsilon_lsst</code>	SuperNNova and periodic-variable broker outputs. LSST rows are safest when timestamped; ZTF rows without event time stay unsafe for early-epoch training.
ANTARES	<code>antares/oracle</code> , <code>antares/superphot_plus</code>	Registered external expert paths; coverage depends on ANTARES backfill availability.
AMPEL	<code>ampel/snguess</code> , <code>ampel/parsnip_followme</code>	State-indexed AMPEL experts. SNGuess is an SN-filter signal; FollowMe is a follow-up-selection signal.
Local physics/features	<code>salt3_chi2</code> , <code>lc_features_bv</code> , <code>oracle_lsst</code>	Local rerunnable physics or feature experts. These are useful because they can be aligned exactly to <code>n_det</code> .

Checked-in result summaries currently report 11 measured trust models in the larger analysis artifacts, with the strongest examples around Fink SNN and the local `lc_features_bv` expert. That should be read as a run artifact, not as a promise that all 28 registered experts are available in every local checkout.

9 How to Run the Project

9.1 Local Smoke Test

If you just want the shortest honest pre-ML audit on a laptop, the one-command path is:

```
python3.11 scripts/run_preml_pipeline.py \  
    --root tmp/preml_demo \  
    --labels data/labels.csv \  
    --lightcurves-dir data/lightcurves \  
    --emit-scoring-snapshots
```

That command runs broker backfill, silver normalization, truth building, local expert inference, gold snapshot construction, the data-readiness audit, and the benchmark check. If you want the manual sequence for debugging, it is still:

```
python3.11 -m pip install -r env/requirements.txt  
python3.11 scripts/download_alerce_training.py --limit 20  
python3.11 scripts/backfill.py --broker all --from-labels data/labels.csv  
python3.11 scripts/normalize.py  
python3.11 scripts/build_truth_table.py --labels data/labels.csv  
python3.11 scripts/build_object_epoch_snapshots.py  
python3.11 scripts/build_expert_helpfulness.py  
python3.11 scripts/train_expert_trust.py  
python3.11 scripts/train_followup.py  
python3.11 scripts/score_nightly.py --from-labels data/labels.csv --n-det 4
```

9.2 BU SCC Execution Model

On SCC, the default trust pipeline is CPU-first. The current `README_scc.md` treats event normalization, truth building, gold-table construction, trust training, follow-up training, scoring, and analysis as CPU stages. SuperNNova can run as a CPU batch inference stage when the expected artifacts are present. GPU time is mainly reserved for ParSNIP or other local experts that actually require it.

9.3 SCC Bootstrap Note

If you are setting this up on SCC, start from `README_scc.md`; it is the more operational document. The short version is:

Important private-repo note. The repo is private. That means `git clone` and `git pull` will fail on SCC unless GitHub authentication is already configured there. The API tokens in `.env` are *not* Git credentials. Lasair and TNS tokens help with data access, but they do not give SCC permission to pull from GitHub.

Before trying the pipeline, make sure one of these is true:

- SCC has an SSH key that is added to your GitHub account, and `origin` uses an SSH URL such as `git@github.com:<org>/<repo>.git`, or
- SCC has HTTPS access to the private repo and you are prepared to authenticate with a GitHub personal access token when prompted.

The practical setup sequence is:

```
# 1. Clone or update the private repository after GitHub auth is working.
cd /projectnb/<yourproject>
git clone <repo-url> rubin_hackathon
cd rubin_hackathon

# 2. Create a project-space virtualenv.
export DEBASS_ROOT=/projectnb/<yourproject>/rubin_hackathon
export DEBASS_VENV=/projectnb/<yourproject>/venvs/debass_meta_env
export DEBASS_PYTHON_MODULE=python3/3.10.12
module load $DEBASS_PYTHON_MODULE
python3 -m venv "$DEBASS_VENV"
source "$DEBASS_VENV/bin/activate"
pip install -r env/requirements.txt

# 3. Add data-access credentials.
cp .env.example .env
# Fill in LASAIR_TOKEN and, if available, the TNS credentials.

# 4. Submit the CPU preparation pipeline.
bash -l jobs/scc_bootstrap.sh --limit 2000
# or, after the environment is already ready:
# bash jobs/submit_cpu_prep.sh --limit 2000
```

After CPU prep, the current SCC guide separates SuperNNova inference and the training chain:

```
# SuperNNova CPU batch stage when artifacts are installed.
qsub -V -P <project> jobs/local_infer_snn.sh

# Then rebuild gold, train trust heads, train follow-up, score, and analyze.
bash jobs/submit_phase1_chain.sh
```

If ParSNIP weights become available and you need GPU inference:

```
qrsh -l gpus=1 -q 140s
cd $DEBASS_ROOT
bash -l jobs/run_gpu_resume.sh --experts=parsnip
```

The operational point is simple: authenticate the private repo first, keep the virtualenv in project space, and do not spend GPU allocation on stages that are already CPU-bound.

CPU side. The main CPU-safe chain is:

```
download_training
-> backfill
-> normalize
-> crossmatch_tns / build_truth_table
-> local_infer_snn when artifacts are installed
```

```
-> build_object_epoch_snapshots
-> build_expert_helpfulness
-> train_expert_trust
-> train_followup
-> score_nightly
```

GPU side. GPU is reserved for local expert reruns that need it, especially ParSNIP. The recommended resume script is `jobs/run_gpu_resume.sh`, which assumes the CPU artifacts already exist.

Why the split matters. The repo is set up so event normalization, gold-table construction, trust training, and follow-up training stay on CPU nodes. GPU only really makes sense for selected local expert inference stages.

10 Tests and What They Tell You

The tests are especially useful here because they often describe the behavior the repo wants, even when the implementation is not fully cleaned up yet.

- `tests/test_no_future_leakage.py`: future expert events must not appear in earlier epochs.
- `tests/test_unsafe_alerce_defaults.py`: unsafe ALerCE snapshot rows are excluded by default.
- `tests/test_projector_fink.py` and `tests/test_projector_lasair.py`: projector logic is source-specific, not one-size-fits-all.
- `tests/test_score_nightly_payload.py`: the final payload must expose `expert_confidence` blocks and an `ensemble` block.
- `tests/test_expert_trust_training.py` and `tests/test_followup_training.py`: the intended training path relies on grouped object splits and on trust features feeding the follow-up head.
- The TNS adapter, truth-table builder, and pre-ML readiness checks have explicit tests, so the newer truth and readiness layer is no longer hand-wavy.

In practice, the tests work like a second specification layer beside the README and the science contract.

11 Current Repository Status on April 30, 2026

Here is the blunt version: the repository is no longer just a pre-ML scaffold. It now has the trust-aware data path, LightGBM training code, scoring payloads, SCC submission scripts, and larger checked-in analysis artifacts. The remaining caution is scientific rather than architectural: truth quality, temporal exactness, and expert availability still determine which claims are safe to make.

11.1 What Is Already Strong

- The science intent is unusually explicit.
- The no-future-leakage lightcurve features are implemented clearly, with 51 feature names in `FEATURE_NAMES`.
- The silver broker-event layer preserves event timing and exactness semantics.
- The gold snapshot builder already encodes the right as-of join behavior.
- The expert registry now covers 28 broker, context, local-rerun, and physics/feature experts.
- The model package is present and the trust-aware training scripts use LightGBM with native NaN handling, regularisation, grouped object splits, and optional calibration.
- There is a one-command pre-ML runner, benchmark checker, readiness audit, expert inventory, and SCC validation workflow.
- TNS crossmatch support exists for building stronger truth when credentials or the bulk CSV are available.
- Local expert coverage has expanded beyond ParSNIP and SuperNNova to include ALERCE-LC, SALT3- χ^2 , Bazin/Villar-style light-curve features, AMPEL/SNGuess, and ORACLE paths.
- The SCC workflow is well documented.

11.2 What Still Blocks the Full Science Goal

The remaining problems are narrower than “the pipeline does not run,” but they still matter:

- The repository contains multiple run artifacts: a tiny local smoke snapshot and larger analysis summaries. Do not mix their metrics without checking which data product generated them.
- Broker-derived weak labels remain weak labels. The canonical truth table must preserve label source and quality, and final science claims need curated or spectroscopic truth where possible.
- Historical ALERCE object probabilities still require care. They can be useful for live scoring and diagnostics, but trust training should exclude `latest_object_unsafe` rows by default.
- Several registered experts are integration targets rather than guaranteed active signals. Coverage depends on credentials, broker history, artifact availability, and survey.
- Local expert outputs still depend on installed artifacts under `artifacts/local_experts/`. In particular, ParSNIP needs both `model.pt` and `classifier.pkl`.

11.3 Checked-In Metrics Snapshot

The current `reports/results\_summary.txt` describes a larger run with 8,774 objects and 139,061 epoch rows. Its truth mix is 4,030 spectroscopic objects and 4,744 weak objects, which is useful progress but still not a license to blur truth provenance. The same summary reports 11 measured trust models and a follow-up model with test ROC-AUC near 0.94. A newer `reports/metrics/followup\_metrics\_safe\_v7.json` reports a follow-up test ROC-AUC near 0.97 for a later artifact. Treat these as run-specific diagnostics until the exact data snapshot, truth policy, and unsafe-row settings are stated alongside the number.

11.4 How To Interpret That Status

For a hackathon teammate, the right way to hold all of this in your head is:

- the data semantics and trust-aware architecture are still the strongest part of the project,
- the ingest, snapshot, training, scoring, and analysis plumbing now exists end to end,
- the next bottleneck is controlled science validation: stronger truth, explicit unsafe-row policy, broker coverage, and real local expert assets.

12 Recommended Reading Order for a New Teammate

If I were onboarding a new teammate tomorrow, this is the reading order I would suggest:

1. README.md
2. docs/science_contract.md
3. src/debass_meta/features/lightcurve.py
4. src/debass_meta/ingest/silver.py
5. src/debass_meta/ingest/gold.py
6. src/debass_meta/projectors/
7. scripts/build_truth_table.py
8. scripts/build_object_epoch_snapshots.py
9. scripts/build_expert_helpfulness.py
10. scripts/local_infer.py
11. README_scc.md
12. jobs/run_gpu_resume.sh

That order moves from the science idea, to epoch-safe features, to event semantics, to the gold training tables, and only then to the operational scripts.

13 Bottom Line

If I had to explain metaDEBASS in one sentence to a teammate, I would call it a trust-aware orchestration layer for a messy set of transient experts. The project is at its best when it preserves timing semantics, exactness labels, and the differences between those experts instead of sanding them down too early. The key move is simple: do not ask only “what class is this object,” ask “which expert should we trust for this object right now.”

As of April 30, 2026, the repository implements the data pipeline, trust training, follow-up training, and scoring layer around that idea. All classifiers use LightGBM with native NaN handling and regularisation, and binary models use class-balance handling plus optional calibration. The remaining work is to turn a working system into a defensible science result: stronger truth, explicit unsafe-row policy, broader broker coverage, and installed local expert artifacts.